

Rethinking Code Complexity Through the Lens of Large Language Models

Chen Xie¹ Xiaodong Gu¹ Yuling Shi¹ Beijun Shen¹

Abstract

Code complexity metrics such as cyclomatic complexity have long been used to assess software quality and maintainability. With the rapid advancement of large language models (LLMs) on coding tasks, an important yet underexplored question arises: *do traditional complexity metrics meaningfully characterize the coding difficulty that LLMs perceive?* In this work, we empirically demonstrate that classical complexity metrics exhibit no consistent correlation with LLM performance, revealing a fundamental mismatch with model-perceived difficulty. To address this gap, we propose LM-CC, a novel code complexity metric tailored for LLMs, grounded in the hypothesis that model-perceived code difficulty is fundamentally driven by semantic nonlinearity. LM-CC quantifies complexity through an entropy-guided semantic compositional hierarchy, capturing the cumulative uncertainty encountered by LLMs during code understanding. Our experimental results demonstrate that LM-CC exhibits strong and consistent partial correlations with LLM performance, while semantics-preserving reductions in LM-CC consistently lead to improved downstream task performance. The source code is available at: <https://github.com/xchen121/lm-cc>.

1. Introduction

Understanding and quantifying code complexity has long been a fundamental problem in software engineering (Weyuker, 1988; Ajami et al., 2019; Feitelson, 2023). Traditional complexity metrics, such as cyclomatic complexity (McCabe, 1976), were originally designed to characterize structural properties of programs, including control-flow diversity, nesting depth, and logical branching. These

¹School of Computer Science, Shanghai Jiao Tong University, Shanghai, China. Correspondence to: Beijun Shen <bjshen@sjtu.edu.cn>.

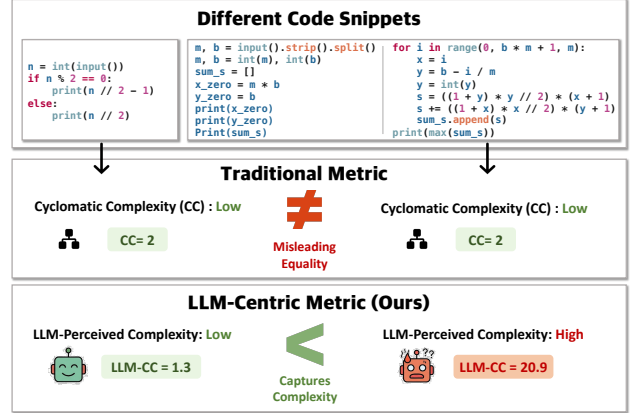


Figure 1. Comparison between Cyclomatic Complexity (CC) and our proposed LM-CC. While CC assigns identical values to code snippets with significantly different cognitive loads for LLMs (top), LM-CC effectively distinguishes them by capturing the model uncertainty on nonlinear code semantics (bottom).

metrics serve as quantitative proxies for the human effort required to comprehend, test, and maintain software systems. Consequently, they have been extensively adopted in a wide range of software engineering applications, including refactoring guidance (Girjoaba & Capiluppi, 2024), maintainability assessment (Benaroch & Lyytinen, 2023), and defect prediction (Alqadi & Maletic, 2020).

Recently, LLMs have achieved remarkable progress across diverse code intelligence tasks, including code completion, generation, summarization, translation, and reasoning (Jiang et al., 2025; Chen et al., 2021; Shi et al., 2025b). Despite these advances, the notion of code complexity itself has rarely been reconsidered from the perspective of LLMs. Existing code complexity metrics primarily characterize program structure from an execution-centric perspective, while largely abstracting away from the representational and inferential mechanisms through which LLMs process code. However, LLMs interpret programs not as executable structures, but as token sequences embedded in high-dimensional semantic spaces. For example, code snippets with identical cyclomatic complexity can elicit markedly different performance from LLMs (Figure 1), indicating a potential mismatch between measured complexity and model-perceived difficulty. This discrepancy raises a critical and underexplored question: *do traditional code complexity metrics*

meaningfully capture the difficulty that LLMs experience when processing code?

Answering this question is nontrivial. LLMs operate on code as linear token sequences encoded in distributed representations, and their difficulty arises from structural nonlinearity across composite and branching semantic units, thereby degrading probabilistic inference (Du et al., 2025; Shi et al., 2025a; Le et al., 2025). Consequently, the aspects of code that challenge LLMs may diverge substantially from those emphasized by traditional complexity measures. A systematic investigation of this potential misalignment is therefore essential for accurately characterizing LLM-perceived code complexity and for informing the design of more effective LLM-based code intelligence methods.

In this paper, we conduct an in-depth empirical study on the relationship between traditional code complexity metrics and LLM performance across multiple code-related tasks. To isolate structural complexity from trivial confounding factors, we perform partial correlation analysis with code length controlled as a covariate. Our results show that commonly used complexity metrics exhibit no statistically reliable correlation with LLM performance once length effects are removed, revealing a fundamental gap between classical complexity measures and LLM-perceived difficulty.

Motivated by this observation, we introduce LM-CC, a novel metric designed to quantify *LLM-perceived Code Complexity*. Our central hypothesis is that the primary source of difficulty for LLMs in code understanding arises from the *nonlinearity of semantic units* inherent in programs. Unlike natural language, which is typically interpreted in a largely sequential fashion, source code exhibits intrinsically nonlinear semantic structures (Anand et al., 2024). Branching, nesting, and scoping introduce discontinuities in semantic flow, requiring models to reason over multiple hierarchically organized semantic paths rather than a single linear context.

Importantly, LLM difficulty is not determined merely by the presence of nonlinear constructs, but by how such constructs compose and interact. While isolated control-flow elements may pose limited challenges, deeply nested or highly branching structures can substantially increase model uncertainty. This suggests that LLM-perceived complexity depends jointly on compositional level and the density of decision points introduced through structural branching.

To capture these properties, we represent source code using a structured semantic abstraction. Specifically, we decompose programs into semantic units based on entropy (i.e., the surprise that LLMs receive during reading) and organize them into a compositional hierarchy that reflects their jumps and dependencies during LLM processing. Based on this representation, LM-CC measures two complementary

dimensions of semantic nonlinearity: *compositional level*, reflecting hierarchical nesting, and *branching factor*, reflecting structural fan-out. Each semantic unit is assigned a local complexity score derived from its compositional level and derived sub-units, and these scores are recursively aggregated across the hierarchy. Intuitively, deeper semantic hierarchies demand more complex multi-level reasoning, while broader branching structures introduce more frequent points of semantic divergence.

We evaluate LM-CC through extensive experiments across diverse code understanding and generation tasks. The results show that LM-CC exhibits strong and consistent partial correlations with LLM performance (Spearman’s r ranging from -0.92 to -0.97) across multiple datasets after controlling for code length, substantially outperforming traditional metrics. Moreover, reducing LM-CC through semantics-preserving code rewriting consistently improves downstream task performance, with gains of up to 20.9%, demonstrating its practical utility as an effective optimization target.

In summary, this paper makes the following contributions:

- We present the first empirical study of code complexity metrics for LLMs, showing that existing metrics fail to reliably capture model-perceived difficulty.
- We propose LM-CC, a novel code complexity metric explicitly grounded in the perspective of large language models.
- We demonstrate the effectiveness and practical utility of LM-CC through extensive experiments on diverse code-related tasks.

2. Revisiting Current Complexity Metrics

We begin by systematically re-examining existing code complexity metrics in the context of LLM-based code tasks. Specifically, we investigate whether these metrics correlate with model performance and thereby reflect model-perceived code difficulty.

2.1. Experimental Setups

Datasets and Tasks. We conduct experiments on two widely used benchmark datasets covering three representative code-related tasks: program repair, code translation, and code execution reasoning. xCodeEval (Khan et al., 2024) is a large-scale multilingual and multitask benchmark for code understanding and generation. From this benchmark, we consider two tasks: (1) *program repair* (xCodeEval-apr), where the model is given buggy code and is required to generate a corrected implementation that passes all test cases; and (2) *code translation* (xCodeEval-ct), where the model translates Python programs into C. Following prior work, we

use filtered subsets consisting of 271 problems for program repair and 535 problems for code translation. We additionally evaluate on HUMAN-EVAL (Chen et al., 2021) under the *code execution reasoning* setting (Liu et al., 2026). In this task, the model is required to infer the correct program output given a code snippet and its corresponding input. The evaluation set contains 162 manually curated programming problems.

Model Configurations. We use DEEPSEEK-V3¹ for all experiments, with task-specific hyperparameters configured according to the settings used in the original papers.

Evaluation Metric. We adopt *pass@1* as the evaluation metric for all tasks. Pass@1 measures the probability that a single model-generated solution correctly solves a given problem by passing all associated test cases.

Correlation Analysis. To assess the effectiveness of existing complexity metrics, we analyze their correlation with LLM performance across tasks. Following Liu et al. (2026), we perform subgroup-based correlation analysis by binning samples into approximately ten equal-sized groups and computing group-level Spearman correlations using median feature values and average task scores. Further details are provided in Appendix C.

We report both zero-order and partial correlations. The zero-order correlation measures the monotonic association between two variables without controlling for confounding factors, and is computed as the Pearson correlation over ranked observations:

$$r_{xy} = \frac{\sum_{i=1}^n (R_i - \bar{R})(S_i - \bar{S})}{\sqrt{\sum_{i=1}^n (R_i - \bar{R})^2} \sqrt{\sum_{i=1}^n (S_i - \bar{S})^2}}, \quad (1)$$

where $R_i = R(x_i)$ and $S_i = R(y_i)$ denote the ranks of variables x and y , and $\bar{R}$ and $\bar{S}$ are their mean ranks. To account for the confounding effect of code length, we further compute the partial correlation coefficient $r_{xy \cdot z}$, which quantifies the association between x and y while controlling for z :

$$r_{xy \cdot z} = \frac{r_{xy} - r_{xz} \cdot r_{yz}}{\sqrt{1 - r_{xz}^2} \cdot \sqrt{1 - r_{yz}^2}}. \quad (2)$$

Complexity Metrics. We assess the following representative code complexity metrics:

- **Cyclomatic Complexity (CC)** (McCabe, 1976): quantifies the number of linearly independent execution paths in a program by analyzing its control-flow graph.
- **Halstead Complexity (HC)** (Halstead, 1977): measures program complexity via the distribution of operators and operands. Among its derived measures, we

Table 1. Correlation between traditional code complexity metrics and pass@1 for code tasks using DEEPSEEK-V3. We report both zero-order correlation and partial correlation while controlling for code length. “-” indicates non-significant results ($p \geq 0.05$).

Metric	Code Task	Correlation	
		Zero-order	Partial
CC	Program Repair	-	-
	Code Translation	-	-0.73
	Execution Reasoning	-0.78	-0.67
HC	Program Repair	-0.97	-0.93
	Code Translation	-0.90	-0.70
	Execution Reasoning	-	-
MI	Program Repair	-	-
	Code Translation	-	-
	Execution Reasoning	-	-0.72
CoC	Program Repair	-0.82	-
	Code Translation	-	-
	Execution Reasoning	-	-

focus on *Halstead Difficulty*, which serves as a proxy for program comprehension effort.

- **Maintainability Index (MI)** (Oman & Hagemeister, 1994): combines Halstead Volume, cyclomatic complexity, lines of code, and comment percentage into a single predictive measure of software maintainability.
- **Cognitive Complexity (CoC)** (Campbell, 2018): estimates code understandability by penalizing nested control structures and nonlinear control flow.

2.2. Preliminary Results

Table 1 summarizes the relationship between traditional code complexity metrics and LLM performance, reporting both zero-order correlations and length-controlled partial correlations. An en dash (“-”) denotes statistically non-significant results. Across all three tasks, none of the existing metrics exhibits stable or consistently significant associations with LLM performance after controlling for code length.

Maintainability Index (MI) achieves a moderate partial correlation ($r = -0.72$) only for the code execution reasoning task, while *Cognitive Complexity (CoC)* exhibits a relatively strong zero-order correlation ($r = -0.79$) in code translation. However, neither metric demonstrates statistically reliable correlations across the remaining tasks, suggesting limited generalizability to model-perceived code difficulty.

Similarly, *Halstead Complexity (HC)* shows strong zero-order correlations in program repair and code translation (-0.97 and -0.90 , respectively), while *Cyclomatic Complexity (CC)* attains a zero-order correlation of -0.78 in execution reasoning. Nevertheless, these correlations substantially weaken once code length is controlled for through partial correlation analysis. Moreover, neither HC nor CC

¹<https://api.deepseek.com>

yields consistently significant partial correlations across tasks, indicating that these metrics fail to robustly capture the structural characteristics that challenge LLM reasoning.

The systematic disappearance of correlations under length control is not incidental. Rather, it reflects a fundamental limitation shared by existing metrics. These metrics are grounded in assumptions about explicit control-flow analysis, human cognitive load, and long-term maintainability, whereas LLMs process programs as token sequences encoded in learned representations. For LLMs, difficulty arises from nonlinear semantic composition and long-range hierarchical dependencies that are not explicitly modeled by traditional complexity measures. This empirical decoupling highlights the need for a fundamentally different notion of code complexity, explicitly aligned with the representational and computational characteristics of large language models.

3. Proposed Metric: LM-CC

3.1. Overview

Motivated by the limitations of existing metrics, we introduce LM-CC, a principled measure of code complexity tailored to large language models. By organizing entropy-aligned semantic units into a hierarchical representation, LM-CC goes beyond surface syntactic abstractions and explicitly encodes the semantic composition and branching relations that govern program semantics and shape LLM uncertainty. While isolated nonlinear constructs are often tractable in isolation, their hierarchical composition induces long-range semantic dependencies and parallel execution contexts, which compound and substantially amplify uncertainty during inference. Consequently, LLM-perceived code complexity cannot be adequately captured by any single structural statistic, but instead arises from the joint interaction between the depth of semantic hierarchies and the density of structurally induced decision points.

Guided by this formulation, we construct LM-CC through a three-stage framework. First, we decompose programs into semantic units via a hybrid entropy–structure mechanism, and organize them into a semantic compositional hierarchy that explicitly encodes nonlinear program semantics (§ 3.2). Second, we extract a set of candidate structural features from this representation—such as compositional level and branching factor—and identify those most predictive of LLM performance through empirical analysis (§ 3.3). Finally, we formalize LM-CC as a weighted aggregation of the selected features (§ 3.4). The complete algorithmic procedure is provided in Appendix A.

Beyond the empirical formulation, we further develop a theoretical interpretation connecting the structure encoded by LM-CC to the inference dynamics of autoregressive language models. This analysis provides insight into why LM-

CC more effectively characterizes LLM-perceived code difficulty than conventional complexity measures (§ 3.5).

3.2. Hierarchical Semantic Decomposition

To characterize code complexity from the perspective of large language models, we propose a structured semantic representation of source code. Programs are decomposed into entropy-aligned semantic units and recursively organized into a semantic compositional hierarchy, enabling the representation to capture how nonlinear program structures induce hierarchical dependencies during LLM inference.

Our decomposition strategy is guided by token-level entropy, which serves as a proxy for model uncertainty. Prior work by Cooper & Scholak (2024) shows that high-entropy tokens correspond to positions where language models exhibit low confidence in next-token prediction, indicating increased processing difficulty. Complementary findings from Shi et al. (2025a) further demonstrate that token-level perplexity effectively identifies semantically salient and structurally important regions in source code. These observations motivate the use of entropy signals to identify semantic boundaries from the perspective of the model.

The hierarchical semantic decomposition proceeds in four steps:

(1) Code preprocessing. We first normalize the source code by removing comments and docstrings, producing a canonical representation that isolates executable program semantics.

(2) Entropy computation. We compute token-level entropy using a pretrained language model. For each token t_i , the entropy is defined as

$$H(t_i) = - \sum_j p(t_j | t_{<i}) \log p(t_j | t_{<i}), \quad (3)$$

where the summation ranges over the entire vocabulary. Higher entropy values indicate greater predictive uncertainty at the corresponding token position.

(3) Semantic unit decomposition. Semantic unit boundaries are identified using a hybrid entropy–structure criterion. A boundary is introduced when either (i) the token entropy exceeds a predefined threshold τ , signaling a local increase in model uncertainty, or (ii) an explicit structural delimiter is encountered, such as the termination of a loop, conditional block, or function definition.

(4) Semantic compositional hierarchy construction. The resulting semantic units are recursively organized into a semantic compositional hierarchy through nested decomposition. Units arising at the same compositional stage form branching structures that encode alternative semantic paths. The final hierarchy therefore captures the nonlinear organi-

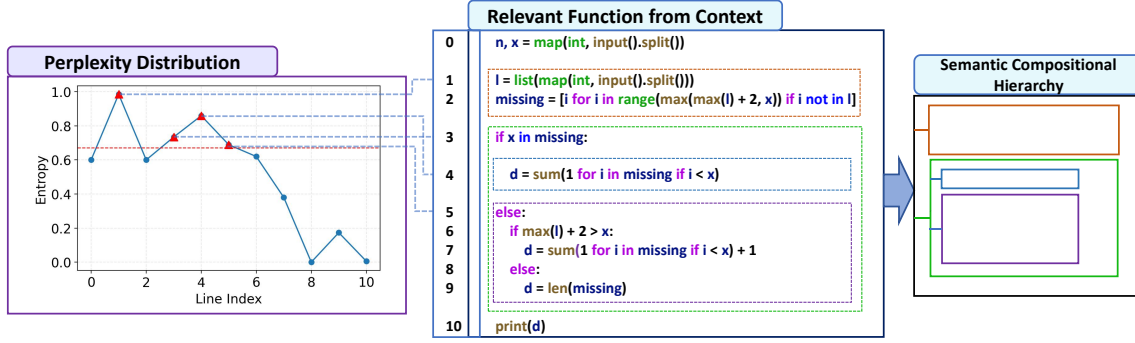


Figure 2. Hierarchical semantic decomposition example. Left: source code with token-entropy annotations, where color-coded regions indicate elevated LLM uncertainty. Right: the induced hierarchical semantic representation, with elements color-aligned to their semantic units of source code.

zation of program semantics in terms of both compositional levels and branching-induced semantic divergence.

Figure 2 illustrates how localized token-level uncertainty is elevated into a structured semantic hierarchy. Rather than treating uncertainty as a flat sequence-level signal, LM-CC integrates entropy signals with structural delimiters to perform hierarchical semantic decomposition, thereby revealing the compositional and branching structures that underlie LLM code processing.

3.3. Feature Extraction and Selection

Based on the semantic compositional hierarchy, we derive a set of quantitative features over semantic units to parameterize LM-CC. These features are designed to characterize two fundamental dimensions of semantic nonlinearity that influence LLM reasoning: *compositional level*, which captures the depth of semantic composition induced by nesting and scoping structures, and *branching factor*, which captures semantic divergence introduced by branching structures.

Specifically, we define the following candidate features:

- *Maximum Compositional Level (MaxCompLevel)*: the maximum compositional level within the hierarchy, representing the deepest nested semantic composition.
- *Average Compositional Level (AvgCompLevel)*: the average compositional level across semantic units, reflecting typical hierarchical complexity.
- *Total Compositional Level (TotalCompLevel)*: the cumulative sum of compositional levels over all semantic units, measuring the overall burden of hierarchical semantic composition.
- *Maximum Branching Factor (MaxBranch)*: the largest branching factor observed in the hierarchy, characterizing the strongest horizontal expansion of semantic alternatives.
- *Average Branching Factor (AvgBranch)*: the average

branching factor across semantic units, quantifying the typical degree of local semantic divergence.

- *Total Branching Factor (TotalBranch)*: the branching factor aggregated over the entire hierarchy, reflecting the global branching complexity.

To examine how these features relate to LLM-perceived difficulty, we perform partial correlation analysis following the protocol described in Section 2, while controlling for code length. We compute group-level Spearman partial correlations between each feature and LLM task performance across multiple code-related tasks (Table 2).

The results reveal clear task-dependent sensitivities to different forms of semantic nonlinearity. For program repair, *TotalCompLevel* exhibits consistently strong negative partial correlations (-0.73 for CodeLlama-7b, -0.87 for Phi-3-mini-4k-instruct, and -0.85 for Qwen2.5-Coder-1.5B), indicating that increasingly deep semantic composition substantially degrades model performance. In contrast, code translation and execution reasoning are jointly influenced by compositional depth and branching complexity. For code translation, both *TotalCompLevel* and *TotalBranch* achieve moderate-to-strong correlations across all evaluated entropy models. Similarly, in execution reasoning, *AvgCompLevel*, *AvgBranch*, and *TotalBranch* all demonstrate statistically significant correlations with model performance.

Despite these strong task-specific effects, no individual feature consistently exhibits robust predictive power across all task-model combinations, as each feature fails to achieve statistical significance under certain settings. To assess overall robustness, we compute the proportion of task-model combinations in which each feature achieves statistical significance. Among all candidates, the global measures *TotalCompLevel* (14/16) and *TotalBranch* (13/16) attain the highest coverage within the two core dimensions of semantic nonlinearity—*compositional level* and *branching factor*, respectively. These results suggest that cumulative semantic

Table 2. Partial Spearman correlations (r) of complexity features and LM-CC with pass@1 across code tasks using DEEPSEEK-V3, while controlling for code length. “-” denotes non-significant correlations ($p \geq 0.05$).

Entropy Model	Code Task	Compositional Level			Branching Factor			LM-CC
		MaxCompLevel	AvgCompLevel	TotalCompLevel	MaxBranch	AvgBranch	TotalBranch	
CodeLlama-7b	Program Repair	-	-	-0.73	-	-	-	-0.93
	Code Translation	-	-	-0.95	-0.85	-	-0.94	-0.97
	Execution Reasoning	-0.72	-0.79	-0.84	-0.69	-0.75	-0.85	-0.92
	Code Summarization	-	-	-0.85	-0.85	-	-0.88	-0.90
Deepseek-Coder-6.7b	Program Repair	-0.80	-	-	-	-	-0.79	-0.88
	Code Translation	-	-	-0.93	-0.86	-	-0.89	-0.94
	Execution Reasoning	-0.75	-0.81	-0.87	-0.72	-0.79	-0.86	-0.93
	Code Summarization	-	-	-0.86	-0.87	-	-0.90	-0.91
Phi-3-mini-4k-instruct	Program Repair	-	-	-0.87	-0.78	-	-	-0.80
	Code Translation	-	-	-0.95	-0.88	-	-0.83	-0.88
	Execution Reasoning	-	-0.77	-	-0.89	-0.81	-0.90	-0.91
	Code Summarization	-	-	-0.92	-0.90	-0.69	-0.95	-0.90
Qwen2.5-Coder-1.5B	Program Repair	-	-0.77	-0.85	-	-	-	-0.87
	Code Translation	-	-	-0.85	-	-0.86	-0.82	-0.95
	Execution Reasoning	-	-0.73	-0.95	-	-0.78	-0.92	-0.84
	Code Summarization	-	-0.73	-0.84	-0.77	-	-0.79	-0.85
Significant Correlation Ratio		3 / 16	6 / 16	14 / 16	11 / 16	6 / 16	13 / 16	16 / 16

composition and global semantic divergence provide the most stable and representative characterization of LLM-perceived code complexity.

3.4. Metric Definition

Building on the complementary yet task-dependent behaviors of the extracted semantic features, we define LM-CC as a unified measure of *LLM-perceived code complexity* that jointly captures the two principal dimensions of semantic nonlinearity: hierarchical semantic composition and branching-induced semantic divergence.

Formally, given a semantic compositional hierarchy, LM-CC is defined as a weighted combination of *TotalBranch* and *TotalCompLevel*. A weighting parameter $\alpha \in [0, 1]$ controls the relative contribution of branching complexity and compositional complexity. The formulation naturally admits an equivalent per-unit decomposition, which makes explicit how local semantic effects accumulate across the hierarchy to produce global complexity.

$$\begin{aligned}
 \text{LM-CC} &= \alpha \cdot \text{TotalBranch} + (1 - \alpha) \cdot \text{TotalCompLevel}, \\
 &= \sum_{v \in V} [\alpha \cdot b(v) + (1 - \alpha) \cdot d(v)],
 \end{aligned}
 \tag{4}$$

where V denotes the set of semantic units in the compositional hierarchy, while $b(v)$ and $d(v)$ denote the branching factor and compositional level associated with unit v , respectively.

From an information-theoretic perspective, compositional level enlarges the contextual span over which branching-induced uncertainty must be propagated, while branching increases the number of competing semantic alternatives

that the model must resolve. LM-CC captures the interaction between these two sources of complexity through a unified weighted formulation, enabling a stable and interpretable characterization of semantic nonlinearity across diverse programming tasks.

Notably, the semantic compositional hierarchy itself is constructed around regions of elevated token entropy, thereby grounding LM-CC in the model’s intrinsic predictive uncertainty. Unlike traditional complexity measures that are derived primarily from syntactic abstractions, LM-CC directly incorporates model-sensitive semantic signals inferred from LLM behavior. Consequently, it provides a more faithful and cognitively aligned representation of code complexity from the perspective of large language models.

3.5. Theoretical Justification

The preceding empirical results show that LM-CC correlates strongly with LLM performance even after controlling for code length, whereas traditional metrics such as cyclomatic complexity (CC) lose predictive power. We now provide a theoretical account for this behavior.

Our central premise is that LLM predictive entropy accumulates systematically along two fundamental dimensions of program semantics: hierarchical semantic composition and branching-induced semantic divergence. These dimensions are explicitly modeled by LM-CC, yet are only weakly characterized by traditional complexity measures such as CC. Formally, CC measures the number of decision points through $\text{CC}(c) = E - N + 2P$ (where E , N , and P denote the numbers of edges, nodes, and connected components of the control-flow graph) without encoding the hierarchy and interaction of program semantics. This distinction is formalized in Proposition B.1 (Appendix B.2), which shows

that two programs with identical CC can exhibit $\Theta(n)$ versus $\Theta(n^2)$ growth in LM-CC depending on whether their semantic structures are flat or deeply nested. This separation explains why CC loses predictive power under length control, whereas LM-CC remains strongly associated with LLM difficulty.

To formalize this intuition, we analyze how predictive entropy accumulates over semantic hierarchies. Our analysis is based on two empirically motivated assumptions. Importantly, the significance of these assumptions lies not primarily in the specific penalty forms themselves, but in the theoretical consequences they enable: namely, the formal separation from CC (Proposition B.1) and the monotonic relationship between LM-CC and structural entropy (Corollary 3.4).

Assumption 3.1 (Depth-Dependent Context Degradation). Predictive uncertainty increases monotonically with compositional depth. For a semantic unit v at depth $d(v)$, the entropy penalty satisfies $\Delta_{\text{depth}}(v) \geq \delta \cdot \max(0, d(v) - d^*)$, where $\delta > 0$ is the per-level degradation rate and d^* is the critical depth beyond which context degradation occurs.

This assumption is supported by empirical findings in long-context LLM reasoning, including the “lost in the middle” phenomenon (Liu et al., 2024), attention sink effects (Xiao et al., 2024), and broader intelligence degradation under extended contexts (Wang et al., 2026b).

Assumption 3.2 (Branching-Induced Uncertainty). At semantic divergence points with branching factor $b \geq 2$, the model incurs additional predictive uncertainty satisfying $\Delta H_{\text{branch}} \geq \gamma \cdot (b - 1)$ for some $\gamma > 0$.

This assumption is motivated by entropy-aware reasoning (Li et al., 2025) and empirical studies consistently showing higher error rates in branching and nested code structures (Sepidband et al., 2025; Basu et al., 2025; Hu et al., 2025). Intuitively, at a branch point the model’s predictive distribution becomes a mixture over semantically divergent continuations, with each additional branch increasing the uncertainty that must be resolved.

Theorem 3.3 (Hierarchical Entropy Accumulation). *Let $\mathcal{T} = (V, E)$ be a semantic hierarchy. Under Assumptions 3.1 and 3.2, the total predictive entropy satisfies*

$$H_{\text{total}}(c) \geq \sum_{v \in V} H_0(v) + \Phi(\mathcal{T}), \quad (5)$$

where $H_0(v)$ is the baseline entropy under ideal context access, and $\Phi(\mathcal{T}) = \delta \sum_v \max(0, d(v) - d^*) + \gamma \sum_v (b(v) - 1)^+$ is the structural penalty.

Proof. For each semantic unit $v \in V$, the conditional entropy decomposes as $H(v) = H_0(v) + \Delta_v$ where $\Delta_v \geq 0$. By Assumption 3.1, units at depth $d(v) > d^*$ incur penalty

$\Delta_{\text{depth}}(v) \geq \delta(d(v) - d^*)$ due to context inaccessibility. By Assumption 3.2, internal units with $b(v) \geq 2$ derived sub-units contribute $\Delta_{\text{branch}}(v) \geq \gamma(b(v) - 1)$. Since these penalties address orthogonal aspects (context span vs. path resolution), aggregating yields $H_{\text{total}} = \sum_v H(v) \geq \sum_v H_0(v) + \Phi(\mathcal{T})$. $\square$

Corollary 3.4 (LM-CC as Structural Complexity Proxy). *The structural penalty satisfies $\Phi(\mathcal{T}) \leq \delta \cdot \text{TotalCompLevel} + \gamma \cdot \text{TotalBranch}$, with equality (up to constants) when all units exceed the critical depth. Thus, $\text{LM-CC} = \alpha \cdot \text{TotalBranch} + (1 - \alpha) \cdot \text{TotalCompLevel}$ is monotonically related to the structural entropy penalty $\Phi(\mathcal{T})$.*

Proof. From $\max(0, d(v) - d^*) \leq d(v)$ and $(b(v) - 1)^+ \leq b(v)$, the upper bound follows. When $d(v) > d^*$ for all v , we have $\Phi = \delta(\text{TotalCompLevel} - d^*|V|) + \gamma(\text{TotalBranch} - |V|)$, establishing the monotonic relationship. $\square$

We note that the additive form provides a conservative lower bound on the true structural penalty; in particular, deeper branching points naturally receive larger contributions through elevated $d(v)$, implicitly reflecting depth-branching interaction. Additional theoretical analyses, including formal separation from cyclomatic complexity and proofs of orthogonality to code length, are provided in Appendix B.

4. Experiments

We evaluate LM-CC through both correlational analysis and causal intervention studies.

4.1. Correlation with Model Performance

We first investigate whether LM-CC effectively characterizes code complexity from the perspective of large language models. Following the datasets and experimental protocol adopted in our empirical evaluation of existing complexity metrics, we compute partial correlation coefficients between LM-CC and LLM task performance while controlling for code length. Token entropy is estimated using four language models spanning different architectures and parameter scales: CODELLAMA-7B-HF², PHI-3-MINI-4K-INSTRUCT³, DEEPSEEK-CODER-6.7B-BASE⁴, and QWEN2.5-CODER-1.5B⁵. The semantic segmentation threshold is fixed at $\tau = 0.67$ following prior work (Wang

²<https://huggingface.co/meta-llama/CodeLlama-7b-hf>

³<https://huggingface.co/microsoft/Phi-3-mini-4k-instruct>

⁴<https://huggingface.co/deepseek-ai/deepseek-coder-6.7b-base>

⁵<https://huggingface.co/Qwen/Qwen2.5-Coder-1.5B>

et al., 2025a), while the weighting factor is set to $\alpha = 0.8$ based on the ablation results presented in Appendix E.2. Additional results obtained with other task-performing LLMs are reported in Appendix E.1.

Overall Effectiveness. As shown in the final column of Table 2, LM-CC exhibits strong and consistent negative correlations with LLM performance across all evaluated tasks and entropy estimation models. Correlation coefficients range from -0.80 to -0.97 , indicating that higher LM-CC values are reliably associated with lower task performance.

Statistical Robustness Across Settings. Although LM-CC does not achieve absolute numerical superiority under every individual setting, results derived from all four entropy estimation models remain consistently statistically significant across all evaluated tasks. Specifically, LM-CC achieves statistically significant correlations in all 16 experimental settings, outperforming its two constituent dimensions, *TotalCompLevel* (14/16) and *TotalBranch* (13/16). These results demonstrate the strong robustness and generalizability of LM-CC across diverse experimental conditions, which is a critical requirement for a reliable evaluation metric.

Cross-Task Generalization. Across diverse code intelligence tasks, LM-CC consistently demonstrates strong negative correlations with model performance, suggesting that it effectively captures the intrinsic difficulty of code as perceived by large language models. Furthermore, parameter settings calibrated on three tasks, namely program repair, code translation, and execution reasoning, transfer effectively to the code summarization task without additional tuning, further validating the strong cross-task generalization ability of LM-CC.

Effect of Entropy Models. The effectiveness of LM-CC varies moderately across entropy estimation models. Under evaluation with DEEPSEEK-V3, CODELLAMA-7B-HF and DEEPSEEK-CODER-6.7B-BASE consistently achieve the strongest overall performance across all tasks, both in terms of correlation strength and statistical effectiveness. In contrast, PHI-3-MINI-4K-INSTRUCT achieves the best performance only on the execution reasoning benchmark, while QWEN2.5-CODER-1.5B does not attain the highest absolute correlation coefficient on that task. We hypothesize that larger base models provide better-calibrated token probability estimates across diverse code patterns, leading to more robust complexity correlations. By comparison, smaller models may lack sufficient capacity to capture fine-grained complexity variations.

Overall, LM-CC demonstrates stable and statistically reliable correlations with LLM performance, substantially outperforming traditional complexity measures. By jointly integrating compositional level and branching factor within an entropy-guided semantic representation, LM-CC pro-

vides a principled characterization of code complexity from the perspective of large language models.

4.2. Impact on LLM-Based Code Tasks

Beyond correlation analysis, we further assess the practical utility of LM-CC through a controlled causal intervention: *Can reducing LM-CC directly improve LLM performance on code-related tasks?* To answer this question, we perform semantics-preserving code rewriting to reduce LM-CC and examine the resulting changes in downstream task performance.

Specifically, we employ DEEPSEEK-V3.2 (DeepSeek-AI, 2024) to rewrite code samples under two constraints: (1) the rewritten program must preserve functional equivalence by passing all original test cases (except for the program repair task), and (2) the rewritten version must achieve a strictly lower LM-CC than the original code. To further disentangle the effect of LM-CC from conventional structural simplifications, we additionally constrain cyclomatic complexity to remain unchanged or increase during rewriting. This ensures that any observed performance gains cannot be attributed to reductions in traditional control-flow complexity measures. Additional details of the rewriting pipeline are provided in Appendix D.

The resulting rewritten dataset therefore contains semantically equivalent programs with systematically reduced LM-CC values while maintaining comparable traditional structural complexity. We subsequently reevaluate all downstream tasks on the subset of samples admitting valid rewrites, using identical evaluation models and experimental settings before and after rewriting. This paired and controlled setup enables direct analysis of the causal impact of reducing LM-CC on LLM performance.

As shown in Table 3, reducing LM-CC consistently yields statistically significant performance improvements across all evaluated tasks. Specifically, program repair, code translation, and execution reasoning achieve relative improvements of 20.9% (from 13.4% to 16.2%), 10.2% (from 42.2% to 46.5%), and 0.96% (from 94.1% to 95.0%), respectively. Notably, the program repair benchmark exhibits the largest improvement after rewriting. This task also has the highest original LM-CC values and the lowest baseline performance, suggesting that semantically complex code imposes substantially greater reasoning difficulty on language models and therefore offers larger room for improvement when complexity is reduced. In contrast, the execution reasoning benchmark consists primarily of relatively short programs with already near-saturated baseline accuracy, naturally limiting the achievable gains from complexity reduction.

Table 3. Impact of semantics-preserving complexity reduction on LLM-based code tasks. Performance (pass@1) is evaluated on the same subset of samples before and after rewriting.

Task	Performance ($\uparrow$)		LM-CC ($\downarrow$)		Cyclomatic ($\downarrow$)	
	Original	Rewritten	Original	Rewritten	Original	Rewritten
Program Repair	13.4%	16.2% [†]	77.7	72.5	18.3	18.3
Code Translation	42.2%	46.5% [†]	45.9	40.9	11.0	11.0
Execution Reasoning	94.1%	95.0% [†]	12.9	9.3	4.1	4.1

5. Related Work

5.1. LLMs for Code

Recent years have witnessed rapid progress in LLMs for code. Codex (Chen et al., 2021) first demonstrated that scaling language models on large code corpora enables strong performance in code generation and comprehension. Subsequently, open-source code LLMs such as Code Llama (Rozière et al., 2023), StarCoder (Lozhkov et al., 2024), DeepSeek-Coder (Zhu et al., 2024), and Qwen2.5-Coder (Hui et al., 2024) achieved competitive performance across diverse coding benchmarks.

Beyond generation, LLMs have been extensively studied on code-centric tasks including program repair (Muenighoff et al., 2023; Shi et al., 2026), code translation (Khan et al., 2024; Wang et al., 2026a), and execution reasoning (Gu et al., 2024). To support systematic evaluation, numerous benchmarks have been introduced, such as HumanEval (Chen et al., 2021), xCodeEval (Khan et al., 2024), LiveCodeBench (Jain et al., 2024), SWE-bench (Jimenez et al., 2024), and BigCodeBench (Zhuo et al., 2025).

While these works demonstrate strong empirical capabilities of LLMs for code, they primarily focus on performance gains and benchmark comparisons, with limited attention to modeling the intrinsic difficulty of code from the perspective of LLMs. This motivates further investigation into the structural factors underlying LLM performance on code tasks.

5.2. Code Complexity Metrics

A broad spectrum of code complexity metrics has been proposed to quantify program difficulty (Gao et al., 2026). Classic examples include *cyclomatic complexity* (McCabe, 1976), which measures the number of linearly independent execution paths via control-flow analysis, and *Halstead’s metrics* (Halstead, 1977), which characterize complexity through the distribution of operators and operands. To better approximate human code comprehension, *Cognitive Complexity* (Campbell, 2018) penalizes nested control structures and control-flow interruptions. Beyond static analyses, recent work has also leveraged neurophysiological and biometric signals, such as EEG-based measures of cognitive

load (Hao et al., 2023) and fMRI-based analyses of brain activation (Peitek et al., 2021), to assess human mental effort during programming tasks.

More recently, several studies have explored the relationship between program complexity and LLM behavior. Sepidband et al. (Sepidband et al., 2025) reported correlations between existing complexity metrics and LLM code generation performance and proposed complexity-aware feedback mechanisms. Other work investigated how program structure and complexity affect LLM execution reasoning (Liu & Jabbarvand, 2025). Related systems such as EpiCoder (Wang et al., 2025b) explicitly control generated code complexity via feature trees, while evaluation frameworks including CodeMind (Liu et al., 2026) and DynaCode (Hu et al., 2025) consistently observe performance degradation as code complexity increases.

Despite these advances, existing studies still rely primarily on traditional complexity metrics and implicitly assume that such measures adequately reflect the difficulty encountered by LLMs. No prior work has introduced a complexity metric explicitly designed to model how LLMs perceive and process code. Our work addresses this gap by proposing a model-aware complexity metric grounded in the representational and computational characteristics of large language models.

6. Conclusion

In this paper, we show that traditional code complexity metrics do not reliably reflect the difficulty that large language models encounter on code-related tasks. To address this issue, we propose LM-CC, a model-aware code complexity metric that uses token-level entropy as a proxy for model uncertainty to quantify semantic nonlinearity induced by the compositional and branching structure of semantic units. We evaluate LM-CC through extensive experiments across diverse code understanding and generation tasks. The results demonstrate that LM-CC correlates substantially more strongly with LLM performance than existing metrics. Furthermore, semantics-preserving transformations that reduce LM-CC consistently improve downstream task performance, highlighting its effectiveness and practical relevance.

Acknowledgements

This research is funded by the National Key Research and Development Program of China (Grant No. 2023YFB4503802), the National Natural Science Foundation of China (Grant No. 62232003), and the Natural Science Foundation of Shanghai (Grant No. 25ZR1401175).

Impact Statement

In this work, we reveal that traditional code complexity metrics are grounded in human cognitive load and exhibit no consistent correlation with LLM performance. In response, we propose LM-CC, a LLM-centric code complexity metric designed to quantify the difficulty LLMs experience when processing code. It decomposes programs into entropy-aligned semantic units, arranges them into a compositional hierarchy, and quantifies complexity by aggregating compositional depth and branching-induced divergence. LM-CC achieves significantly stronger and more consistent correlations with task performance than traditional metrics, providing a unified model-aware code complexity standard.

References

- Ajami, S., Woodbridge, Y., and Feitelson, D. G. Syntax, predicates, idioms - what really affects code complexity? *Empir. Softw. Eng.*, 24(1):287–328, 2019.
- Alqadi, B. S. and Maletic, J. I. Slice-based cognitive complexity metrics for defect prediction. In *27th IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2020, London, ON, Canada, February 18-21, 2020*, pp. 411–422. IEEE, 2020.
- Anand, A., Verma, S., Narasimhan, K., and Mezini, M. A critical study of what code-LLMs (do not) learn. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 15869–15889. Association for Computational Linguistics, 2024.
- Basu, K., Abdelaziz, I., Kate, K., Agarwal, M., Crouse, M., Rizk, Y., Bradford, K., Munawar, A., Kumaravel, S., Goyal, S., et al. Nestful: A benchmark for evaluating llms on nested sequences of api calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33526–33535, 2025.
- Benaroch, M. and Lyytinen, K. How much does software complexity matter for maintenance productivity? the link between team instability and diversity. *IEEE Trans. Software Eng.*, 49(4):2459–2475, 2023.
- Campbell, G. A. Cognitive complexity: A new way of measuring understandability. White paper, SonarSource SA, 2018. URL <https://www.sonarsource.com/docs/CognitiveComplexity.pdf>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Cooper, N. A. and Scholak, T. Perplexed: Understanding when large language models are confused. *arXiv preprint arXiv:2404.06634*, 2024.
- DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Du, Y., Tian, M., Ronanki, S., Rongali, S., Bodapati, S. B., Galstyan, A., Wells, A., Schwartz, R., Huerta, E. A., and Peng, H. Context length alone hurts LLM performance despite perfect retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 23281–23298. Association for Computational Linguistics, 2025.
- Feitelson, D. G. From code complexity metrics to program comprehension. *Commun. ACM*, 66(5):52–61, 2023.
- Gao, H., Hijazi, H., Medeiros, J., Durães, J., Lam, C., de Carvalho, P., and Madeira, H. Complementarity in software code complexity metrics. *J. Syst. Softw.*, 232: 112679, 2026.
- Girjoaba, A. V. and Capiluppi, A. Refactoring legacy code using cleaning up cycles: An experience report. In *IEEE International Conference on Software Maintenance and Evolution, ICSME 2024, Flagstaff, AZ, USA, October 6-11, 2024*, pp. 753–764. IEEE, 2024.
- Gu, A., Rozière, B., Leather, H., Solar-Lezama, A., Synnaeve, G., and Wang, S. CRUXEval: A benchmark for code reasoning, understanding and execution. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 16568–16621. PMLR, 2024.
- Halstead, M. H. *Elements of Software Science*. Operating and Programming Systems Series. Elsevier Science Inc., 1977.
- Hao, G., Hijazi, H., Durães, J., Medeiros, J., Couceiro, R., Lam, C. T., Teixeira, C., Castelhana, J., Castelo-Branco, M., Carvalho, P., et al. On the accuracy of code complexity metrics: A neuroscience-based guideline for improvement. *Frontiers in Neuroscience*, 16:1065366, 2023.
- Hu, W., Duan, J., Wei, C., Zhang, L., Zhang, Y., and Xu, K. DynaCode: A dynamic complexity-aware code benchmark for evaluating large language models in code generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 21980–21997. Association for Computational Linguistics, 2025.

- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., et al. Qwen2.5-Coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.
- Jain, N., Han, K., Gu, A., Li, W.-D., Yan, F., Zhang, T., Wang, S., Solar-Lezama, A., Sen, K., and Stoica, I. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Jiang, J., Wang, F., Shen, J., Kim, S., and Kim, S. A survey on large language models for code generation. *ACM Transactions on Software Engineering and Methodology*, 2025.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Khan, M. A. M., Bari, M. S., Do, X. L., Wang, W., Parvez, M. R., and Joty, S. XCodeEval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6766–6805. Association for Computational Linguistics, 2024.
- Le, C. C., Vinh, H. C. T., Phan, H. N., Le, D. D., Nguyen, T. N., and Bui, N. D. Q. VisualCoder: Guiding large language models in code execution with fine-grained multimodal chain-of-thought reasoning. In *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, volume NAACL 2025 of *Findings of ACL*, pp. 6628–6645. Association for Computational Linguistics, 2025.
- Li, X., Callanan, E., Ghassel, A., and Zhu, X. Entropy-gated branching for efficient test-time reasoning. *arXiv preprint arXiv:2503.21961*, 2025.
- Liu, C. and Jabbarvand, R. A tool for in-depth analysis of code execution reasoning of large language models. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, pp. 1178–1182. ACM, 2025.
- Liu, C., Chen, Y., and Jabbarvand, R. CodeMind: Evaluating large language models for implicit and explicit code execution reasoning. *IEEE Transactions on Software Engineering*, 2026.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Lozhkov, A., Li, R., Allal, L. B., Cassano, F., Lamy-Poirier, J., Tazi, N., Tang, A., Pykhtar, D., Liu, J., Wei, Y., et al. StarCoder 2 and The Stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.
- McCabe, T. J. A complexity measure. *IEEE Transactions on Software Engineering*, (4):308–320, 1976.
- Muennighoff, N., Liu, Q., Zebaze, A., Zheng, Q., Hui, B., Zhuo, T. Y., Singh, S., Tang, X., von Werra, L., and Longpre, S. OctoPack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124*, 2023.
- Oman, P. and Hagemester, J. Construction and testing of polynomials predicting software maintainability. *Journal of Systems and Software*, 24(3):251–266, 1994.
- Peitek, N., Apel, S., Parnin, C., Brechmann, A., and Siegmund, J. Program comprehension and code complexity metrics: An fMRI study. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 524–536, 2021.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Remez, T., Rapin, J., et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Sepidband, M., Taherkhani, H., Wang, S., and Hemmati, H. Enhancing LLM-based code generation with complexity metrics: A feedback-driven approach. In *49th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2025, Toronto, ON, Canada, July 8-11, 2025*, pp. 1416–1426. IEEE, 2025.
- Shi, Y., Qian, Y., Zhang, H., Shen, B., and Gu, X. Long-CodeZip: Compress long context for code language models. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, November 16-20, 2025*, pp. 141–153. IEEE, 2025a.
- Shi, Y., Zhang, H., Wan, C., and Gu, X. Between lines of code: Unraveling the distinct patterns of machine and human programmers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 51–62. IEEE Computer Society, 2025b.
- Shi, Y., Wang, S., Wan, C., Wang, M., and Gu, X. From code to correctness: Closing the last mile of code generation with hierarchical debugging. In *2026 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2026.

- Wang, C., Yu, T., Xie, C., Wang, J., Chen, D., Zhang, W., Shi, Y., Gu, X., and Shen, B. EvoC2Rust: A skeleton-guided framework for project-level c-to-rust translation. In *2026 IEEE/ACM 47th International Conference on Software Engineering (ICSE) SEIP*, 2026a.
- Wang, S., Yu, L., Gao, C., Zheng, C., Liu, S., Lu, R., Dang, K., Chen, X., Yang, J., Zhang, Z., et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. *arXiv preprint arXiv:2506.01939*, 2025a.
- Wang, W., Min, J., and Zou, W. Intelligence degradation in long-context llms: Critical threshold determination via natural length distribution analysis. *arXiv preprint arXiv:2601.15300*, 2026b.
- Wang, Y., Li, H., Zhang, X., Wu, J., Liu, X., Hu, W., Guo, Z., Huang, Y., Xin, Y., Yang, Y., Su, J., Chen, Q., and Li, S. EpiCoder: Encompassing diversity and complexity in code generation. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. OpenReview.net, 2025b.
- Weyuker, E. J. Evaluating software complexity measures. *IEEE Trans. Software Eng.*, 14(9):1357–1365, 1988.
- Xiao, G., Tian, Y., Chen, B., Han, S., and Lewis, M. Efficient streaming language models with attention sinks. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.
- Zhu, Q., Guo, D., Shao, Z., Yang, D., Wang, P., Xu, R., Wu, Y., Li, Y., Gao, H., Ma, S., et al. DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024.
- Zhuo, T. Y., Vu, M. C., Chim, J., Hu, H., Yu, W., Widyasari, R., Yusuf, I. N. B., Zhan, H., He, J., Paul, I., Brunner, S., Gong, C., Hoang, T., Zebaze, A. R., Hong, X., Li, W.-D., Kaddour, J., Xu, M., Zhang, Z., Yadav, P., Jain, N., Gu, A., Cheng, Z., Liu, J., Liu, Q., Wang, Z., Lo, D., Hui, B., Muennighoff, N., Fried, D., Du, X., de Vries, H., and von Werra, L. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *Proceedings of the 13th International Conference on Learning Representations*, 2025.

Appendix

A. Pseudocode of LM-CC

Algorithm 1 presents the computation procedure of LM-CC. Given a source code snippet, LM-CC first preprocesses the input and computes token-level entropy using a pretrained language model (Lines 1–4). Semantic unit boundaries are then identified using two complementary signals: positions where model uncertainty exceeds the threshold τ , and syntactic delimiters such as loops or function boundaries (Lines 5–9). This hybrid criterion allows the segmentation to capture both model-perceived difficulty and program structure. The algorithm subsequently constructs a semantic compositional hierarchy through a BFS traversal (Lines 10–15). Starting from a top-level unit, each unit range is recursively partitioned according to detected boundaries and indentation structure, and the resulting sub-units are enqueued for further processing. After constructing the hierarchy, LM-CC is computed by aggregating contributions from all semantic units (Lines 16–19), where each unit is weighted according to its branching factor and compositional level. Consequently, semantically deeper and more branching structures receive higher complexity scores.

Algorithm 1 Computation of LM-CC

Input: Source code P , entropy threshold τ , smoothing constant α

Output: Complexity score $\text{LM-CC}(P)$

```

1  $P' \leftarrow \text{REMOVECOMMENTS}(P)$   $(t_1, \dots, t_n) \leftarrow \text{TOKENIZE}(P')$ 
2 foreach token  $t_i$  do
3    $H(t_i) \leftarrow -\sum_j p(t_j \mid t_{<i}) \log p(t_j \mid t_{<i})$ 
4 end
5  $\mathcal{B} \leftarrow \emptyset$  foreach position  $i$  do
6   if  $H(t_i) > \tau$  or  $t_i$  is syntactic boundary then
7     Mark  $i$  as unit boundary
8   end
9 end
10 Partition  $P'$  into semantic units  $\mathcal{B} = \{b_1, \dots, b_m\}$ 
11  $T \leftarrow$  empty hierarchy with top unit  $r$   $Q \leftarrow$  queue initialized with  $(r, 0, |\mathcal{B}| - 1)$  while  $Q$  is not empty do
12    $(v, s, e) \leftarrow Q.\text{DEQUEUE}()$  Partition semantic units  $\mathcal{B}[s : e]$  by boundaries and indentation foreach contiguous unit range  $[s', e']$  do
13     Create compositional sub-unit  $u$  for units  $\mathcal{B}[s' : e']$  Add  $u$  as lower-level unit of  $v$   $Q.\text{ENQUEUE}((u, s', e'))$ 
14   end
15 end
16  $\text{LM-CC}(P) \leftarrow 0$  foreach unit  $v \in T$  do
17    $\text{LM-CC}(P) \leftarrow \text{LM-CC}(P) + \alpha \cdot b(v) + (1 - \alpha) \cdot d(v)$ 
18 end
19 return  $\text{LM-CC}(P)$ 

```

B. Extended Theoretical Analysis

This section extends the theoretical analysis presented in Section 3.5. We first introduce the formal setup and notation (§ B.1), and then present additional theoretical results on the separation of LM-CC from traditional complexity measures (§ B.2) and the justification for entropy-driven semantic segmentation (§ B.3).

B.1. Formal Setup and Notation

Let $\mathcal{M}$ be an autoregressive language model processing a code token sequence $c = (t_1, t_2, \dots, t_n)$. The predictive entropy at position i is $H_i := H(t_i \mid t_{<i}) = -\sum_{v \in \mathcal{V}} P_{\mathcal{M}}(v \mid t_{<i}) \log P_{\mathcal{M}}(v \mid t_{<i})$, where $\mathcal{V}$ denotes the vocabulary.

A semantic unit $u = (t_l, \dots, t_r)$ is a maximal contiguous subsequence satisfying: (i) for all $i \in (l, r)$, $H_i \leq \tau$ and t_i is not a syntactic delimiter; and (ii) either $H_l > \tau$, $l = 1$, or t_{l-1} is a syntactic delimiter. The collection of semantic units partitions the code sequence.

A semantic hierarchy $\mathcal{T} = (V, E)$ represents the compositional organization of program semantics, where each element $v \in V$ corresponds to a semantic unit and relations in E encode syntactic composition between units. The compositional level of a unit v , denoted by $d(v)$, reflects its degree of nested composition, while the branching factor $b(v)$ quantifies the number of semantic units produced from the same compositional context. Let $L(v)$ denote the token length of unit v . We

define the unit-level conditional entropy as $H(v) := \sum_{i=l}^r H(t_i | t_{<i})$, where unit v spans the token sequence $(t_l, \dots, t_r)$.

Supporting Evidence for Assumptions. Assumption 3.1 (Depth-Dependent Context Degradation) is supported by extensive empirical evidence. Liu et al. (2024) demonstrate a “lost in the middle” phenomenon where language models exhibit significantly degraded performance when relevant information is positioned in the middle of long contexts, with a characteristic U-shaped performance curve favoring information at context boundaries. Furthermore, studies on attention patterns reveal that transformers allocate disproportionate attention to initial tokens (“attention sinks”) (Xiao et al., 2024), and that effective context utilization degrades with distance even within the nominal context window. Recent work on intelligence degradation in long-context LLMs (Wang et al., 2026b) further confirms that model performance deteriorates progressively as context length increases beyond certain thresholds.

Assumption 3.2 (Branching Uncertainty) is motivated by recent findings on entropy-aware reasoning in language models. Studies on test-time compute (Li et al., 2025) show that high-entropy tokens correspond to critical decision points where prediction uncertainty is elevated, and that concentrating computation budget on such high-uncertainty moments improves reasoning quality. In the context of code, control-flow constructs (conditionals, loops) introduce structural ambiguity requiring the model to reason about multiple execution paths. Empirical studies confirm that LLM-generated code with deeper nesting and more comparisons exhibits higher error rates (Sepidband et al., 2025), and benchmarks specifically designed for nested structures (Basu et al., 2025; Hu et al., 2025) consistently observe performance degradation as nesting complexity increases.

B.2. Separation from Traditional Metrics

We establish that LM-CC captures semantic properties fundamentally different from cyclomatic complexity (CC).

Proposition B.1 (Separation from Cyclomatic Complexity). *There exist code families $\{c_n^{\text{flat}}\}$ and $\{c_n^{\text{nested}}\}$ such that: (i) $CC(c_n^{\text{flat}}) = CC(c_n^{\text{nested}}) = n + 1$; (ii) $|c_n^{\text{flat}}| = \Theta(|c_n^{\text{nested}}|)$; and (iii) $\text{LM-CC}(c_n^{\text{flat}}) = \Theta(n)$ while $\text{LM-CC}(c_n^{\text{nested}}) = \Theta(n^2)$.*

Proof. **Construction of c_n^{flat} .** Consider n sequential independent conditionals at the same nesting level:

```
if cond1: A1 else: B1
if cond2: A2 else: B2
...
if condn: An else: Bn
```

This code has n decision points, so $CC = n + 1$ by McCabe’s formula. For LM-CC, the code decomposes into $1 + n + 2n = 3n + 1$ semantic units: one top-level unit, n conditional units, and $2n$ branch bodies $(A_1, B_1, \dots, A_n, B_n)$, at compositional levels 1, 2, and 3 respectively. Thus $\text{TotalCompLevel} = 1 + 2n + 6n = \Theta(n)$ and $\text{TotalBranch} = \Theta(n)$, yielding $\text{LM-CC}(c_n^{\text{flat}}) = \Theta(n)$.

Construction of c_n^{nested} . Consider n nested conditionals:

```
if cond1:
    if cond2:
        ...
        if condn: An else: Bn
        ...
    else: B2
else: B1
```

This code also has n decision points, so $CC = n + 1$. For LM-CC, the code decomposes into $2n + 1$ semantic units: n conditional units at compositional levels 1, 2, $\dots$, n , $n - 1$ else-branches $(B_1, \dots, B_{n-1})$ each at level $i + 1$ for the i -th conditional, and two terminal units (A_n, B_n) at level $n + 1$. Thus $\text{TotalCompLevel} = \Theta(n^2)$ and $\text{TotalBranch} = \Theta(n)$, yielding $\text{LM-CC}(c_n^{\text{nested}}) = \Theta(n^2)$.

Code length. The flat structure has length $O(n)$. The nested structure has length $O(n)$ for the code statements plus $O(n^2)$ for indentation whitespace. Since complexity metrics typically operate on token counts (not raw characters) and indentation tokens are often normalized, the effective lengths are $\Theta(n)$ in both cases. $\square$

This separation has practical implications: code with identical cyclomatic complexity can exhibit vastly different LLM processing difficulty depending on hierarchical organization. By Theorem 3.3, deeper hierarchies induce greater entropy accumulation, so the $\Theta(n)$ vs. $\Theta(n^2)$ gap in LM-CC reflects actual differences in LLM difficulty that CC cannot capture.

Proposition B.2 (Orthogonality to Code Length). *For any code length L , there exist code samples c_1 and c_2 with $|c_1| = |c_2| = L$ such that $|\text{TotalCompLevel}(\mathcal{T}_1) - \text{TotalCompLevel}(\mathcal{T}_2)| = \Omega(L)$, where $\mathcal{T}_1$ and $\mathcal{T}_2$ denote the corresponding semantic hierarchies.*

Proof. We construct two code samples of length L tokens, each partitioned into $k = \Theta(\sqrt{L})$ semantic units of average length $\Theta(\sqrt{L})$.

Flat structure (c_1). All k units reside at compositional level 1, yielding $\text{TotalCompLevel}(\mathcal{T}_1) = k = \Theta(\sqrt{L})$.

Chain structure (c_2). Units are arranged in a linear compositional chain with levels $1, 2, \dots, k$, yielding $\text{TotalCompLevel}(\mathcal{T}_2) = \sum_{i=1}^k i = k(k+1)/2 = \Theta(L)$.

Thus, $|\text{TotalCompLevel}(\mathcal{T}_1) - \text{TotalCompLevel}(\mathcal{T}_2)| = \Theta(L) - \Theta(\sqrt{L}) = \Omega(L)$. $\square$

These results explain why cyclomatic complexity loses predictive power under length control: CC counts decision points without encoding their hierarchical arrangement, and empirically $\text{CC}(c) \approx \beta_0 + \beta_1|c|$, so length control eliminates the signal. In contrast, LM-CC captures hierarchical organization—a dimension orthogonal to length.

B.3. Entropy-driven Segmentation

We provide theoretical justification for using entropy thresholds to identify semantic unit boundaries.

Proposition B.3 (Entropy Thresholding as Boundary Detection). *Suppose the entropy sequence $\{H_i\}_{i=1}^n$ exhibits a bimodal structure: tokens within semantic units have entropy concentrated around a low level μ_L , while boundary tokens have entropy concentrated around a high level $\mu_H > \mu_L$, each with noise bounded by η . That is, $H_i \in [\mu_L - \eta, \mu_L + \eta]$ for interior tokens and $H_i \in [\mu_H - \eta, \mu_H + \eta]$ for boundary tokens. If the gap satisfies $\mu_H - \mu_L > 2\eta$, then choosing threshold $\tau = (\mu_L + \mu_H)/2$ correctly classifies all tokens as interior or boundary.*

Proof. For interior tokens: $H_i \leq \mu_L + \eta < (\mu_L + \mu_H)/2 = \tau$ (since $\mu_H - \mu_L > 2\eta$ implies $\mu_L + \eta < \mu_H - \eta$ and thus $\mu_L + \eta < (\mu_L + \mu_H)/2$).

For boundary tokens: $H_i \geq \mu_H - \eta > (\mu_L + \mu_H)/2 = \tau$ (by symmetric argument).

Thus, the threshold τ achieves perfect separation under the bimodal assumption. $\square$

Remark B.4. In practice, the threshold τ is selected empirically (e.g., using a quantile of the observed entropy distribution) rather than computed from unknown population parameters. Following Wang et al. (2025a), we set τ at the 67th percentile of token entropies, which provides robust boundary detection across diverse code samples.

This result formalizes the intuition that entropy spikes mark semantic boundaries. Empirical studies of token-level entropy in code (Cooper & Scholak, 2024; Shi et al., 2025a) confirm that code entropy exhibits approximately bimodal behavior, with elevated values at control-flow transitions, function boundaries, and other structural discontinuities, justifying the threshold-based segmentation approach.

C. Subgroup-based Correlation Analysis

To assess the correlation between code complexity metrics and LLM performance while mitigating sensitivity to individual samples, we adopt a subgroup-based correlation analysis. Specifically, samples are first ranked according to the feature or metric of interest and then partitioned into approximately ten equal-sized groups.

Within each group, we take the median feature (or metric) value as the representative statistic, which provides robustness against outliers and extreme cases in code structure or length. For task performance, we use the mean score within each group, as the median may degenerate to binary values (e.g., all 0 or all 1) in groups corresponding to particularly difficult or trivial instances, thereby obscuring performance variation.

We then compute the Spearman rank correlation coefficient r between the group-level feature values and performance scores. To further mitigate the effects of grouping sensitivity and sample-level randomness, we repeat the analysis with the number of groups ranging from 9 to 11, and report the statistically significant result ($p < 0.05$) with the largest absolute correlation magnitude. This procedure yields more stable and reliable correlation estimates by attenuating the influence of individual sample variance and grouping artifacts.

D. Semantics-Preserving Code Rewriting for LM-CC Reduction

We conduct semantics-preserving code rewriting to directly reduce LM-CC and examine its causal impact on model performance. Specifically, we instruct an LLM to simplify each code example, and compute LM-CC and cyclomatic complexity (CC) before and after rewriting. We retain only samples for which CC remains unchanged while LM-CC decreases, ensuring that structural complexity is held constant. Semantic equivalence is verified using test cases: rewritten samples for code translation and execution reasoning are required to pass all tests, whereas those for program repair are required to fail the tests, consistent with task definitions. We employ DEEPSEEK-V3.2 for code rewriting with $n=10$ attempts and a temperature of 1. The prompt used for rewriting is provided below.

Prompt for Code Rewriting

You are an expert in Python programming.

Instruction:

- Improve the code’s comprehensibility and readability without altering its original functionality.
- Do not modify the function’s cyclomatic complexity (e.g., do not add or remove loops or conditional branches).
- Output only the simplified code inside ``python`` without any additional explanations.

Initial Code:

{code}

Simplified Code:

To exclude simple samples with limited simplification potential, we further filter the samples where the original LM-CC ranked in the top 50% (and in the top 60% for the execution reasoning task). This yields valid subsets of 58, 99, and 60 samples for program repair, code translation, and execution reasoning, respectively.

E. Additional Experimental Results

This section provides supplementary experimental analyses that further validate the effectiveness, robustness, and practical properties of LM-CC. We first report additional evaluation results using GPT-4o-mini and Qwen3.5-122B as task-performing models (§ E.1), followed by ablation studies examining the contribution of the hierarchical decomposition strategy and the weighting factor α (§ E.2). Finally, we present a cost analysis of LM-CC and discuss its computational efficiency in practical deployment settings (§ E.3).

E.1. Supplementary Results on GPT-4o-mini and Qwen3.5-122B

In addition to DEEPSEEK-V3, we further evaluate LM-CC on two additional task-performing models, GPT-4O-MINI and QWEN3.5-122B, using the same experimental setup as described in the main text. The results, summarized in Table 4, are highly consistent with those reported in Section 4, further supporting the effectiveness and generalizability of LM-CC.

Specifically, LM-CC achieves a perfect 16/16 significant correlation rate across all evaluated settings, consistently outperforming its two component dimensions, TotalCompLevel and TotalBranch. Nevertheless, the effectiveness of different entropy estimation models varies across tasks and task-performing LLMs. When GPT-4O-MINI is adopted, DEEPSEEK-CODER-6.7B-BASE and PHI-3-MINI-4K-INSTRUCT achieve the strongest numerical performance across most tasks, whereas CODELLAMA-7B-HF and QWEN2.5-CODER-1.5B perform competitively only on the code translation task. In contrast, when QWEN3.5-122B is employed, nearly all entropy models obtain strong results on program repair and code translation, but exhibit relatively weak numerical performance on the execution reasoning task.

Table 4. Partial Spearman correlations (r) of complexity features and LM-CC with pass@1 across code tasks using GPT-4O-MINI and QWEN3.5-122B, while controlling for code length. “-” indicates non-significant correlations ($p \geq 0.05$).

Entropy Model	Code Task	Compositional Level			Branching Factor			LM-CC
		MaxCompLevel	AvgCompLevel	TotalCompLevel	MaxBranch	AvgBranch	TotalBranch	
GPT-4O-MINI								
CodeLlama-7b	Program Repair	-	-	-0.93	-	-	-0.83	-0.85
	Code Translation	-0.67	-	-0.93	-0.92	-0.80	-0.97	-0.97
	Execution Reasoning	-	-0.96	-0.94	-	-0.83	-0.92	-0.94
	Code Summarization	-0.81	-0.97	-0.99	-1.0	-	-0.99	-0.99
Phi-3-mini-4k-instruct	Program Repair	-	-0.75	-0.88	-	-	-0.69	-0.9
	Code Translation	-	-	-0.95	-0.95	-0.95	-0.98	-0.98
	Execution Reasoning	-	-0.93	-0.95	-0.76	-0.64	-0.93	-0.98
	Code Summarization	-0.95	-	-0.99	-1.0	-	-0.99	-0.99
Qwen2.5-Coder-1.5B	Program Repair	-	-0.80	-0.92	-	-	-	-0.80
	Code Translation	-	-0.75	-0.94	-0.91	-0.88	-0.96	-0.98
	Execution Reasoning	-0.71	-0.90	-0.88	-	-	-0.87	-0.87
	Code Summarization	-0.81	-0.93	-0.99	-1.0	-	-0.99	-0.99
deepseek-coder-6.7b-base	Program Repair	-	-0.83	-0.73	-	-	-0.77	-0.91
	Code Translation	-	-	-0.94	-0.94	-0.88	-0.93	-0.95
	Execution Reasoning	-0.72	-0.96	-0.93	-0.92	-0.82	-0.98	-0.98
	Code Summarization	-0.92	-0.95	-0.99	-1.0	-	-0.99	-0.99
Significant Correlation Ratio		7/16	11/16	16/16	10/16	7/16	15/16	16/16
QWEN3.5-122B								
CodeLlama-7b	Program Repair	-	-	-0.82	-	-	-0.85	-0.98
	Code Translation	-0.85	-	-0.90	-0.77	-	-0.85	-0.93
	Execution Reasoning	-	-0.72	-0.79	-	-0.82	-0.72	-0.79
	Code Summarization	-0.87	-0.92	-0.99	-1.0	-	-0.99	-0.99
Phi-3-mini-4k-instruct	Program Repair	-0.68	-0.75	-0.95	-0.79	-	-	-0.97
	Code Translation	-	-	-0.88	-0.82	-0.88	-0.84	-0.88
	Execution Reasoning	-0.71	-	-0.81	-0.89	-0.80	-0.80	-0.76
	Code Summarization	-0.95	-0.74	-0.99	-0.97	-	-0.99	-0.99
Qwen2.5-Coder-1.5B	Program Repair	-0.85	-0.68	-0.92	-	-	-	-0.98
	Code Translation	-	-	-0.76	-0.69	-	-0.75	-0.77
	Execution Reasoning	-	-0.71	-0.80	-	-0.83	-0.82	-0.80
	Code Summarization	-0.9	-0.87	-0.99	-0.93	-	-0.99	-0.99
deepseek-coder-6.7b-base	Program Repair	-	-	-	-	-	-0.90	-0.98
	Code Translation	-	-	-0.88	-0.74	-	-0.78	-0.79
	Execution Reasoning	-	-0.84	-0.84	-	-	-0.80	-0.71
	Code Summarization	-0.95	-0.88	-0.99	-0.98	-	-0.99	-0.96
Significant Correlation Ratio		8/16	9/16	15/16	10/16	4/16	14/16	16/16

E.2. Ablation Study

Ablation on the Hierarchical Decomposition Strategy. The entropy-driven decomposition strategy in LM-CC integrates token-level entropy with syntax-aware structural delimiters to construct a semantic compositional hierarchy over source code. To systematically evaluate the effectiveness and necessity of these two design elements, we conduct an ablation study by removing each one in turn. The results are presented in Table 5.

The results show that removing either entropy-guided boundaries (*syntax-only*) or syntax-guided hierarchical structuring (*entropy-only*) consistently reduces the stability of the correlation, and under certain experimental settings, the resulting coefficients fail to reach statistical significance. In contrast, LM-CC, which jointly incorporates both design elements, achieves more stable and consistently competitive performance across configurations.

It should be noted that while depth and branching are well-established in traditional code complexity analysis, our key contribution is redefining them from an LLM-centric perspective. Traditional metrics use syntax-driven structures and reflect human-oriented complexity. In contrast, LM-CC derives semantic units via entropy measurements and computes depth and branching scores over this LLM-aligned hierarchy, thereby capturing both the inherent uncertainty of LLM predictions and the compositional nonlinearity of code processing. Our ablation experiments confirm that syntax-based baselines with the same aggregation logic achieve inferior performance, proving that the improvement originates from LLM-driven hierarchical decomposition rather than the choice of complexity dimensions.

Table 5. Ablation results on decomposition strategies for entropy and syntactic delimiter. “-” indicates non-significant correlations ($p \geq 0.05$). PR, CT, and ER stand for the three tasks of program repair, code translation, and execution reasoning, respectively.

Strategy	DeepSeek-V3			GPT-4o-mini			Qwen3.5-122b		
	PR	CT	ER	PR	CT	ER	PR	CT	ER
Syntax-only	-0.88	-0.78	-	-0.85	-0.93	-0.78	-0.99	-0.93	-0.71
Entropy-only	-0.72	-0.95	-	-0.93	-0.99	-0.90	-	-0.84	-
LM-CC	-0.93	-0.97	-0.92	-0.85	-0.97	-0.94	-0.98	-0.93	-0.79

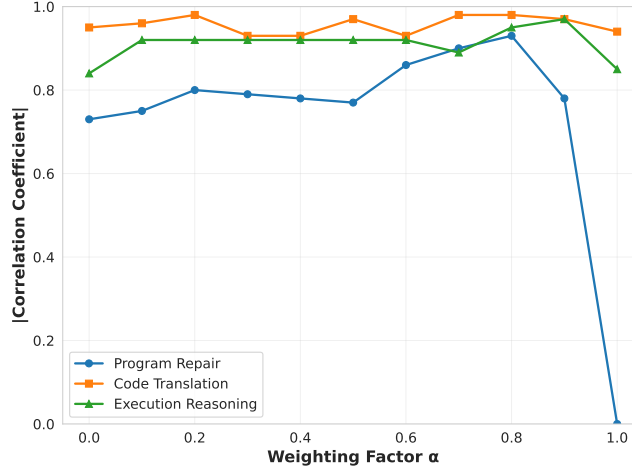


Figure 3. Ablation on the weighting factor α in LM-CC. Performance peaks at intermediate α values, while hierarchy-only ($\alpha \rightarrow 0$) and branching-only ($\alpha \rightarrow 1$) configurations perform substantially worse.

Ablation on the Weighting Factor α . To investigate the impact of the weighting factor α , we conduct an ablation study by computing LM-CC under a range of α values. The results are shown in Figure 3. As α increases, the effectiveness of LM-CC remains relatively stable for code translation. For execution reasoning, it exhibits a mild rise followed by a gradual decline with limited variation, whereas for program repair it shows an increase followed by a sharper drop, indicating substantially higher sensitivity to α . Based on these observations, we set $\alpha = 0.8$, which yields consistently strong performance across all three tasks and strikes a favorable balance between the two structural features captured by LM-CC.

E.3. Cost Analysis

LM-CC computes token-level entropy via a single forward pass, eliminating the need for autoregressive decoding. As a result, its computational complexity is linear in the input sequence length ($O(n)$ tokens). Table 6 summarizes the average per-sample computation time for both Cyclomatic Complexity (CC) (0.1–0.9 ms/sample) and LM-CC (0.02–0.7 s/sample) across all four evaluation tasks. Although LM-CC incurs higher latency than lightweight static analysis metrics, it remains operationally feasible. It supports full parallelization via mini-batching and delivers far more expressive semantic representations, making it an attractive choice for large-scale empirical studies.

F. Implications of LM-CC

The proposed LM-CC enables several potential applications in LLM-centered software engineering and AI-assisted programming:

LLM-Aware Code Evaluation and Benchmark Design: LM-CC allows benchmarks and datasets to be stratified according to difficulty as perceived by large language models. By explicitly controlling for LLM-perceived complexity, it enables more faithful assessment of model capabilities, supports fairer cross-model comparisons, and facilitates finer-grained analysis of structural failure modes that are not captured by code length or traditional complexity measures.

Table 6. Comparison of average per-sample computation time between CC and LM-CC across different code tasks.

Task	Avg LoC	Avg Time/Sample	
		CC	LM-CC
Program Repair	61.0	0.9ms	0.7s
Code Translation	20.8	0.4ms	0.46s
Execution Reasoning	8.4	0.1ms	0.02s
Code Summarization	16.9	0.2ms	0.44s

LLM-Guided Code Refactoring and Simplification: LM-CC provides a principled signal for semantics-preserving code transformations that target reductions in LLM-perceived complexity. In contrast to traditional refactoring, which primarily optimizes for human readability or maintainability, model-aware refactoring restructures semantic compositional patterns to improve LLM performance without altering program behavior. This is particularly relevant for tasks such as program repair, code translation, code execution reasoning, and code understanding.

Adaptive Reasoning and Tool Invocation: LM-CC can serve as a control signal for dynamically selecting reasoning strategies in LLM-based systems. Code segments with higher perceived complexity may trigger structured prompting, explicit step-by-step reasoning, or external tool invocation (e.g., execution or symbolic analysis), whereas simpler code can be handled with lightweight generation, enabling cost- and performance-aware orchestration.

Training Data Curation and Curriculum Learning: By quantifying code difficulty from the model’s perspective, LM-CC can guide the construction of training curricula that better align with LLM-specific processing challenges. Ordering or weighting samples by model-aware complexity may improve training efficiency, robustness, and generalization compared to curricula based solely on syntactic properties or code length.

Failure Analysis and Model Diagnosis: By explicitly capturing structural sources of difficulty such as deep composition and high branching intensity of semantic units, LM-CC supports systematic diagnosis of LLM failures on code tasks. It helps explain performance degradation on complex programs—even when they are short—offering interpretable insights into how program semantic composition interacts with model behavior.